2P Actividad 2 SOA en Spring-boot

TEAMS

Nombre:

Ricardo Fuentes Arredondo

Gabriel Guerrero Romero

Matrícula:

870256154

830185752

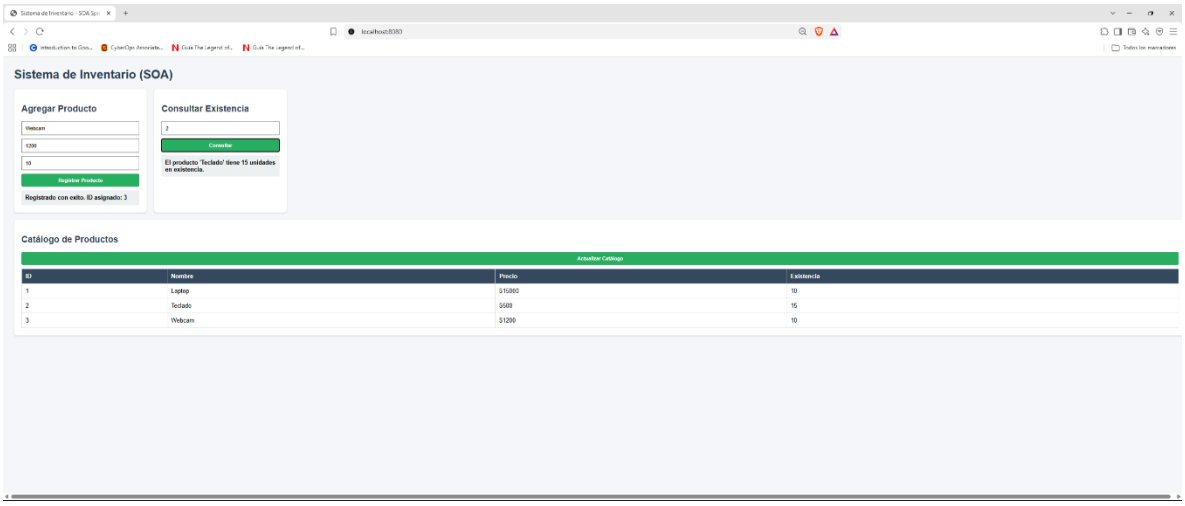
Materia:

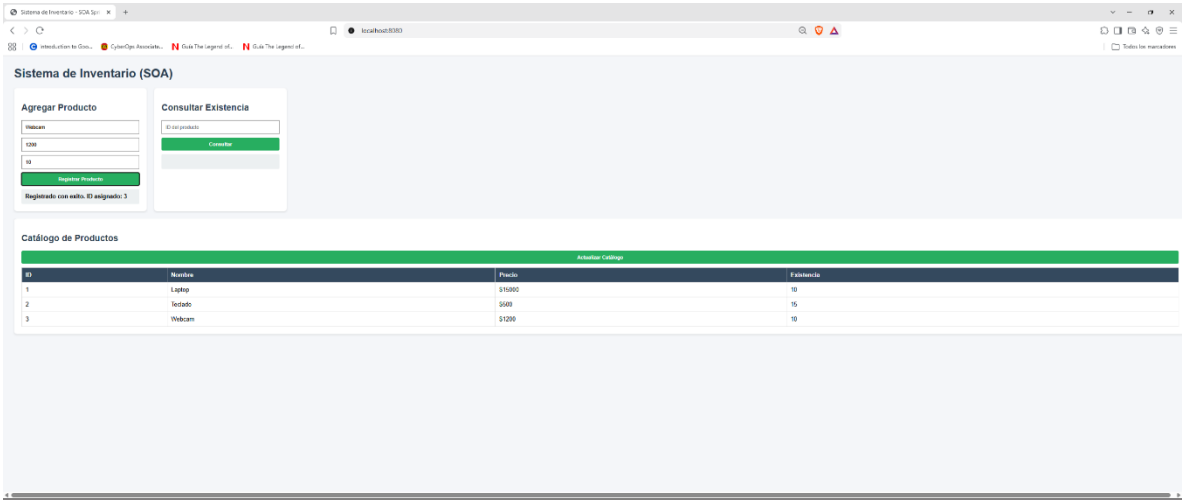
Tecnología de Construcción de Servicios Web

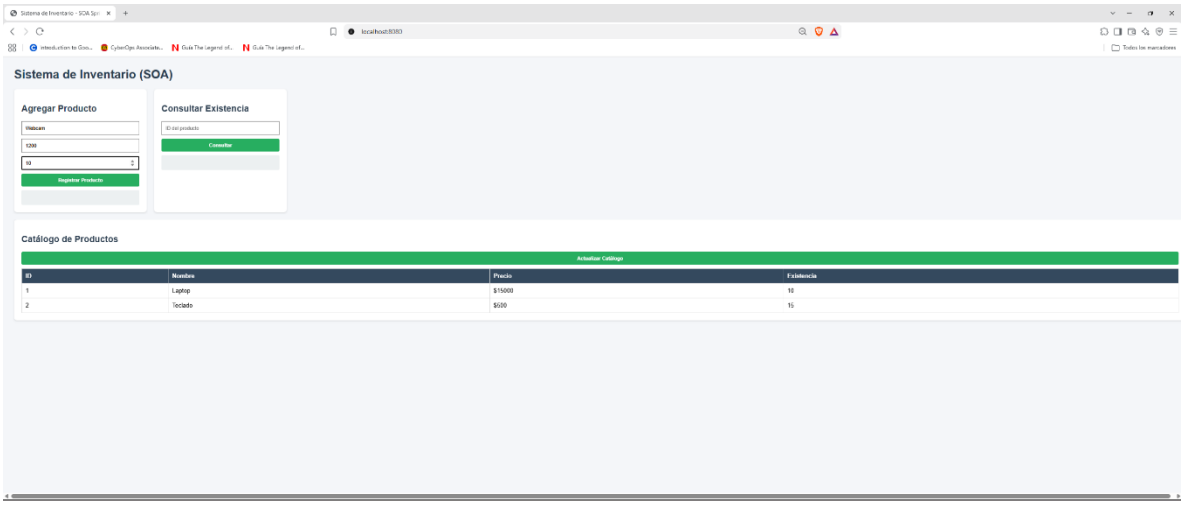
Profesor:

Yazmin Villarreal Vivian

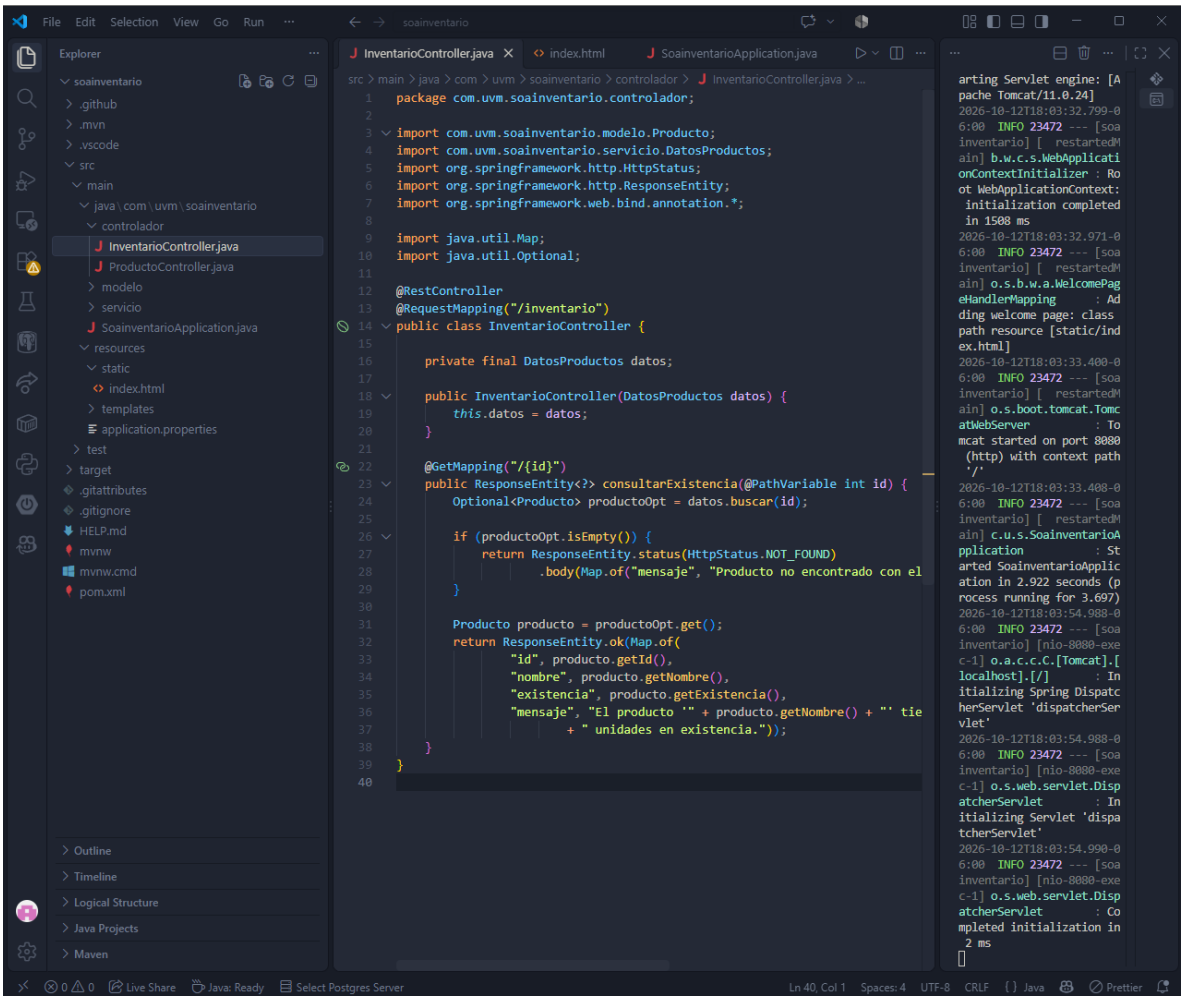
Capturas del navegador

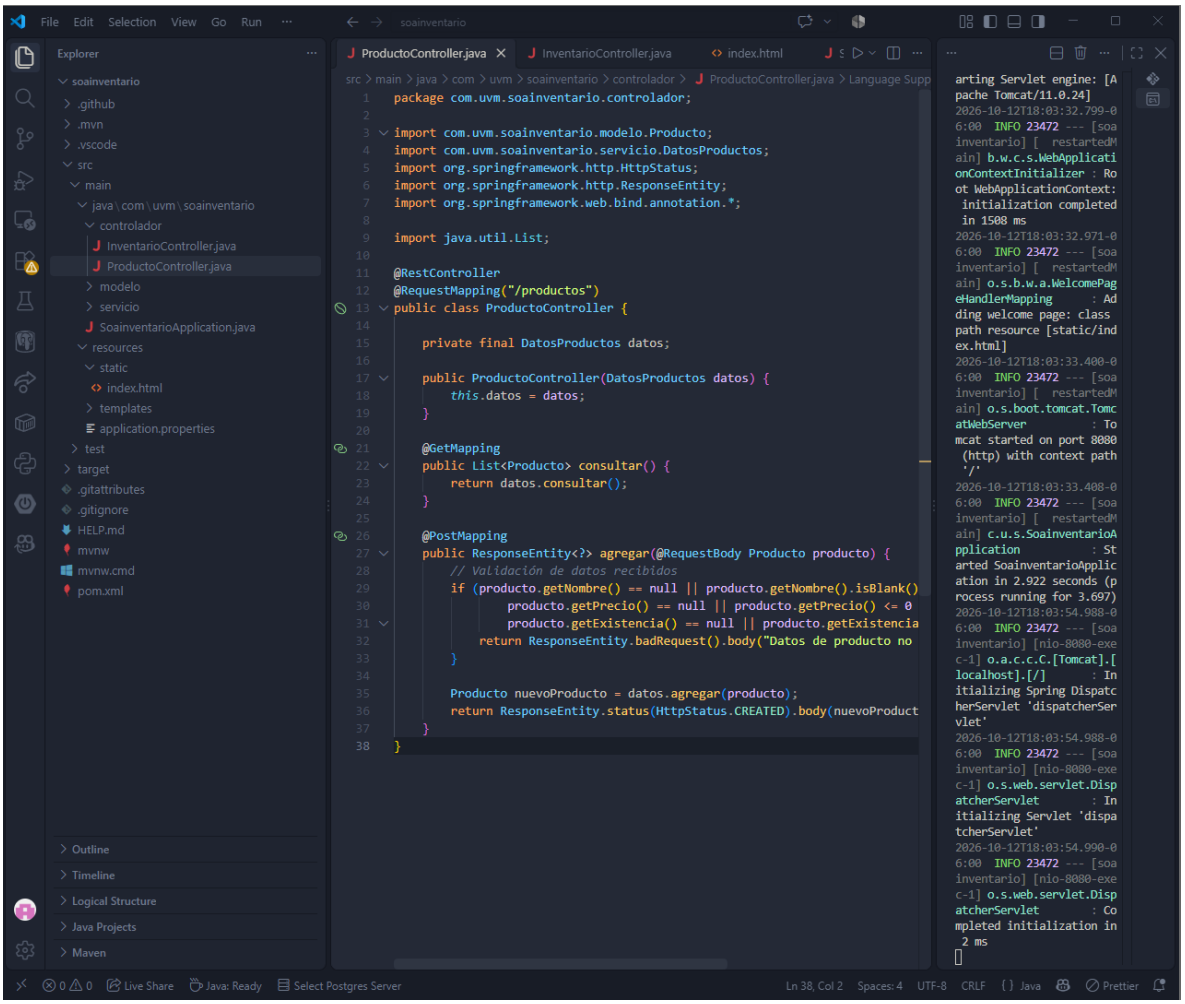


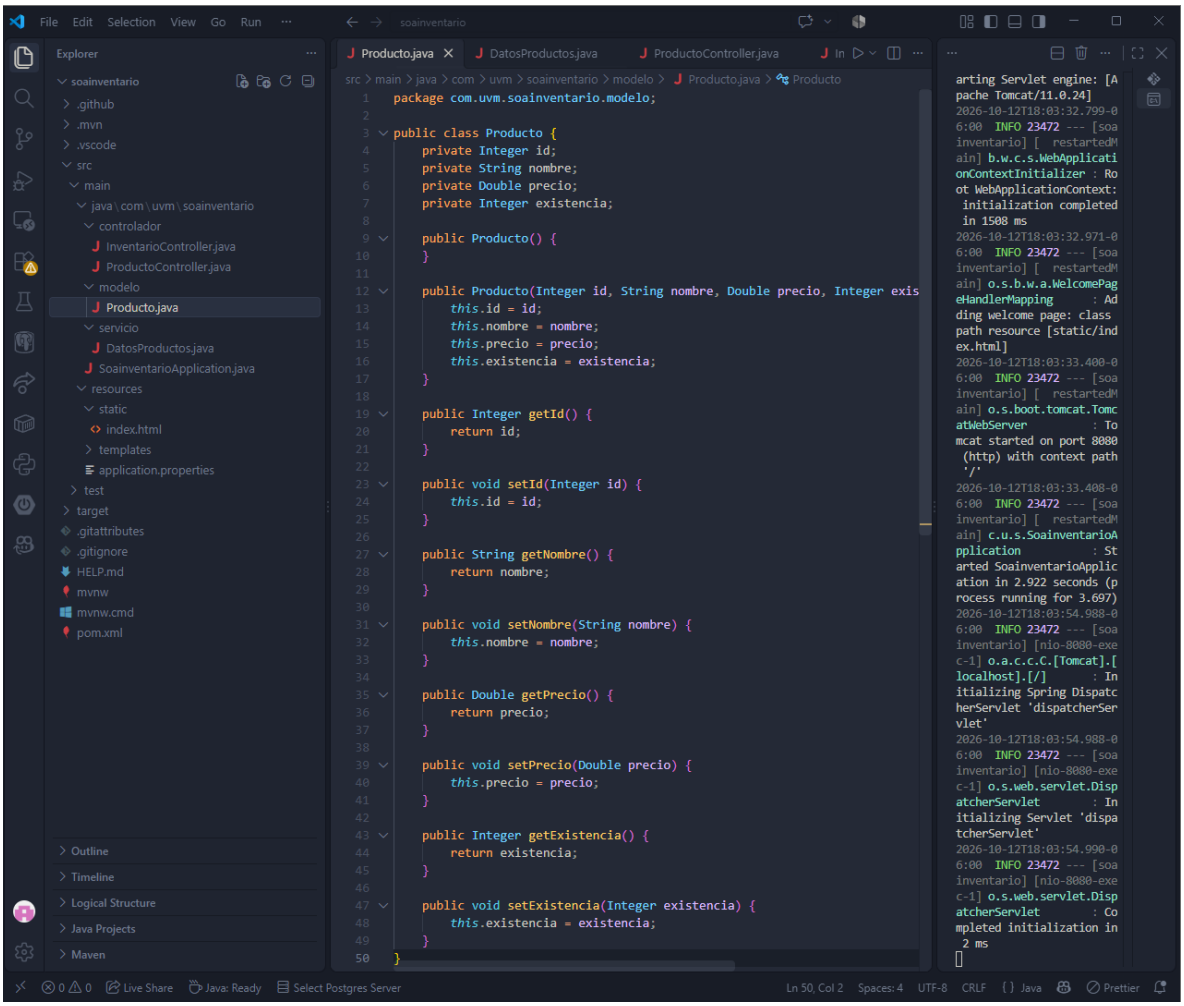


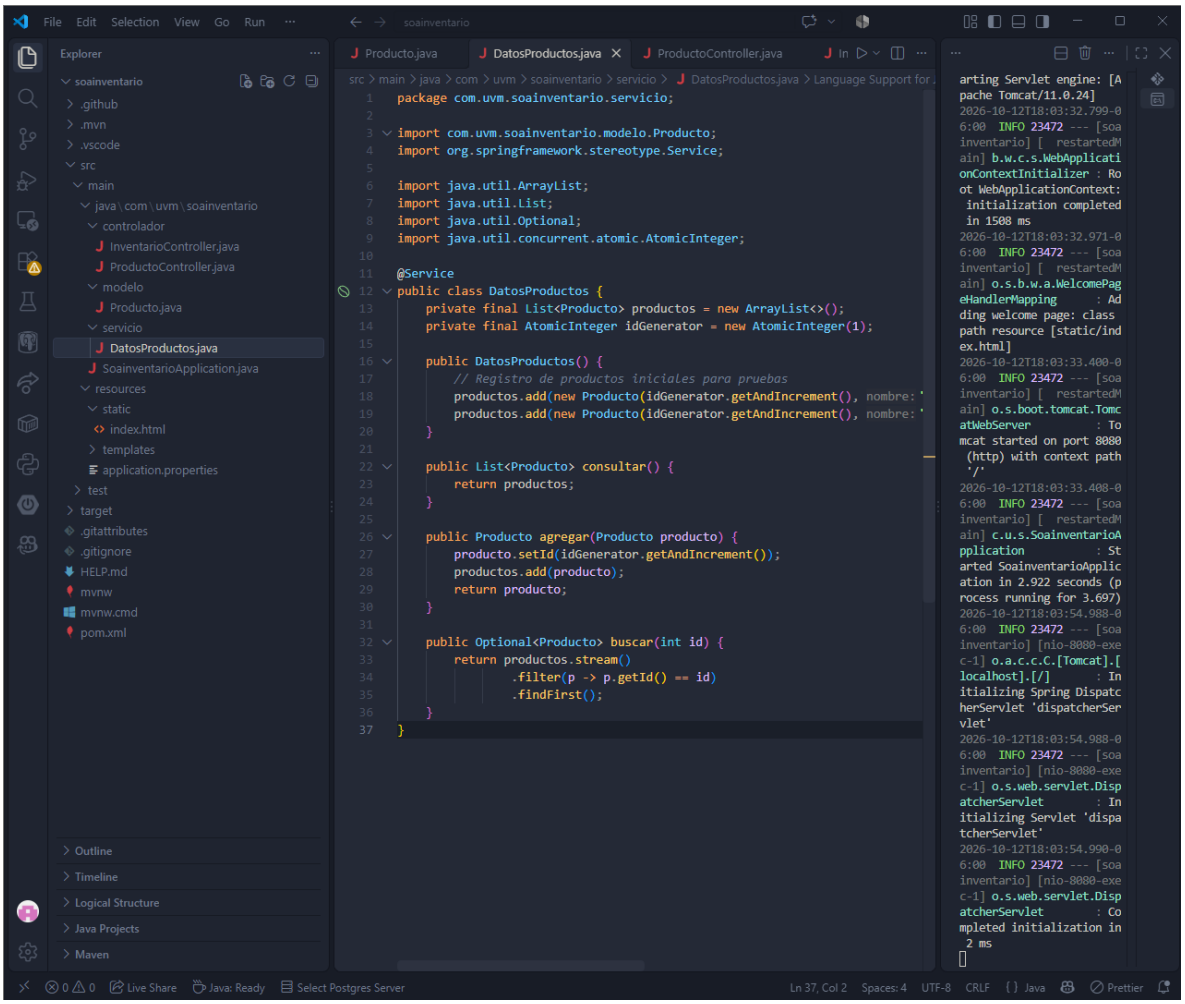


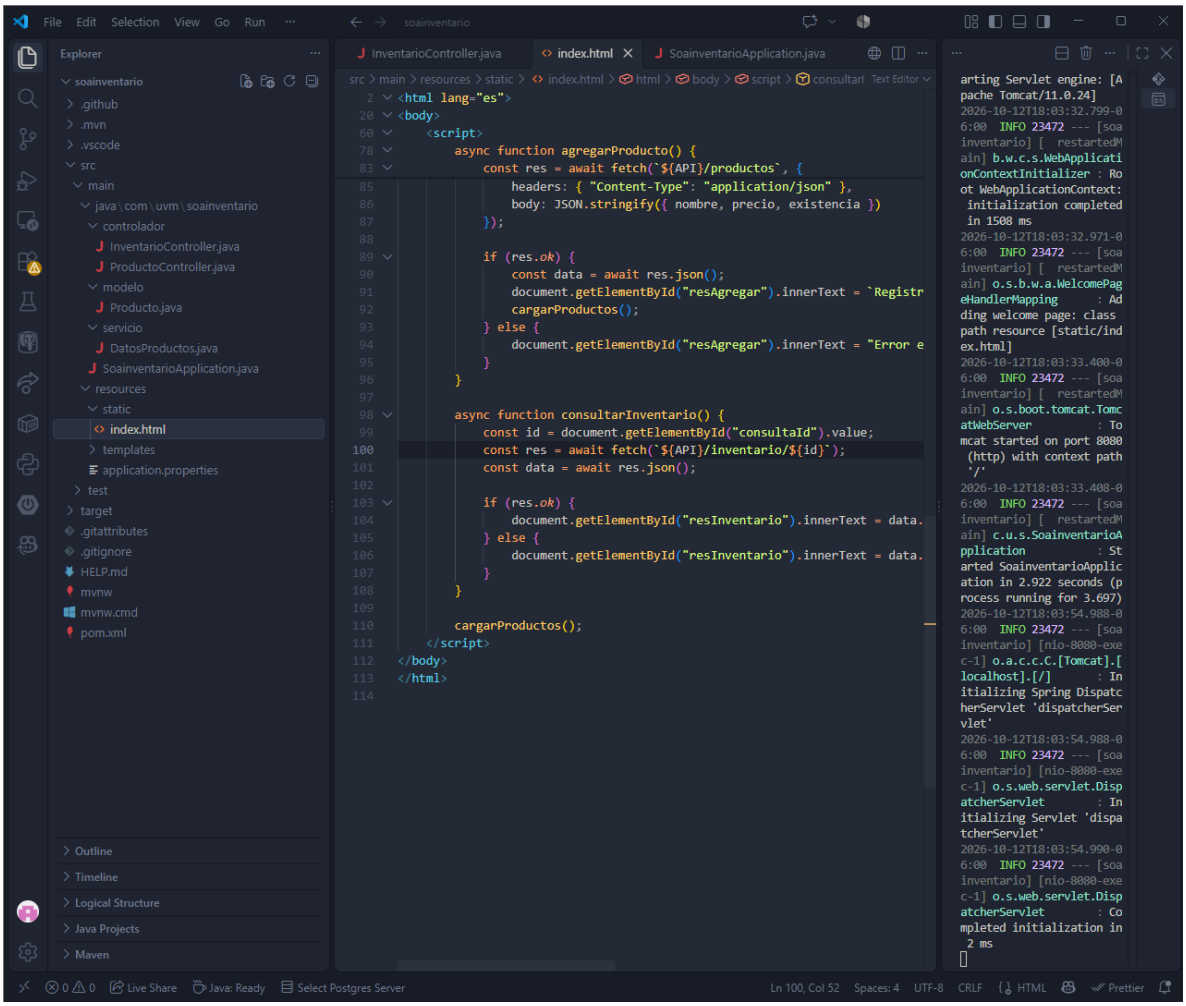
Capturas de los códigos

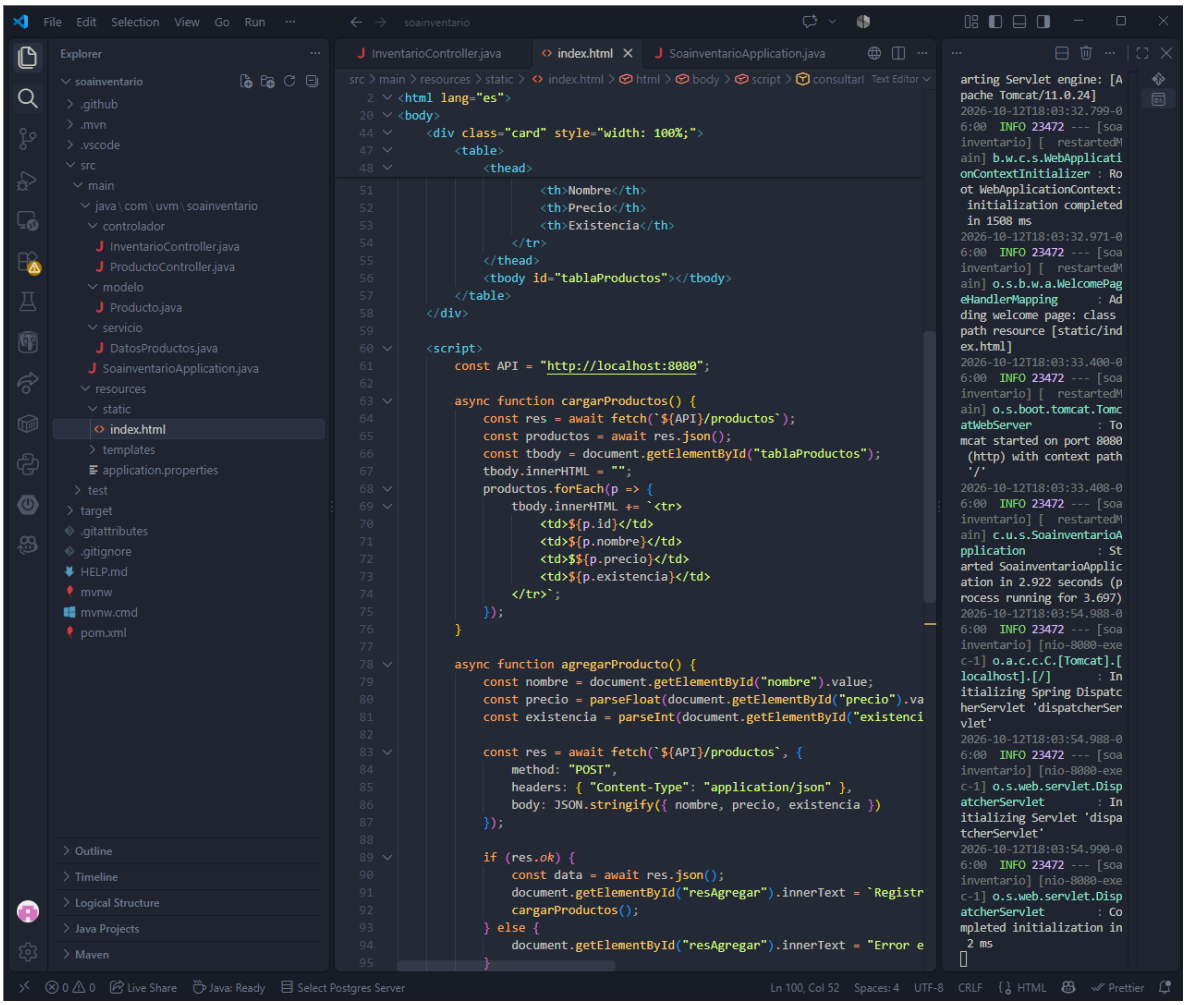

File Edit Selection View Go Run
soainventario
Explorer
soainventario
.github
.mvn
.vscode
src
main
java \ com \ uvm \ soainventario
controlador
InventarioController.java
ProductoController.java
modelo
servicio
SoainventarioApplication.java
resources
static
index.html
templates
application.properties
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
Outline
Timeline
Logical Structure
Java Projects
Maven
InventarioController.java
index.html
SoainventarioApplication.java
src > main > java > com > uvm > soainventario > controlador > InventarioController.java > ...
1 package com.uvm.soainventario.controlador;
2
3 import com.uvm.soainventario.modelo.Producto;
4 import com.uvm.soainventario.servicio.DatosProductos;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.Map;
10 import java.util.Optional;
11
12 @RestController
13 @RequestMapping("/inventario")
14 public class InventarioController {
15
16 private final DatosProductos datos;
17
18 public InventarioController(DatosProductos datos) {
19 this.datos = datos;
20 }
21
22 @GetMapping("/{id}")
23 public ResponseEntity<?> consultarExistencia(@PathVariable int id) {
24 Optional<Producto> productoOpt = datos.buscar(id);
25
26 if (productoOpt.isEmpty()) {
27 return ResponseEntity.status(HttpStatus.NOT_FOUND)
28 .body(Map.of("mensaje", "Producto no encontrado con el
29 }
30
31 Producto producto = productoOpt.get();
32 return ResponseEntity.ok(Map.of(
33 "id", producto.getId(),
34 "nombre", producto.getNombre(),
35 "existencia", producto.getExistencia(),
36 "mensaje", "El producto '" + producto.getNombre() + "' tie
37 + " unidades en existencia."));
38 }
39 }
40
arting Servlet engine: [Apache Tomcat/11.0.24]
2026-10-12T18:03:32.799-06:00 INFO 23472 --- [soainventario] [restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1508 ms
2026-10-12T18:03:32.971-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.b.w.a.WelcomePageHandlerMapping : Adding welcome page: class path resource [static/index.html]
2026-10-12T18:03:33.400-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-10-12T18:03:33.408-06:00 INFO 23472 --- [soainventario] [restartedMain] c.u.s.SoainventarioApplication : Started SoainventarioApplication in 2.922 seconds (process running for 3.697)
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-10-12T18:03:54.990-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
Ln 40, Col 1 Spaces: 4 UTF-8 CRLF Java Prettier


File Edit Selection View Go Run
soainventario
Explorer
soainventario
.github
.mvn
.vscode
src
main
java \ com \ uvm \ soainventario
controlador
InventarioController.java
ProductoController.java
modelo
servicio
SoainventarioApplication.java
resources
static
index.html
templates
application.properties
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
Outline
Timeline
Logical Structure
Java Projects
Maven
ProductoController.java
InventarioController.java
index.html
src > main > java > com > uvm > soainventario > controlador > ProductoController.java > Language Supp
package com.uvm.soainventario.controlador;
import com.uvm.soainventario.modelo.Producto;
import com.uvm.soainventario.servicio.DatosProductos;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/productos")
public class ProductoController {
private final DatosProductos datos;
public ProductoController(DatosProductos datos) {
this.datos = datos;
}
@GetMapping
public List<Producto> consultar() {
return datos.consultar();
}
@PostMapping
public ResponseEntity<?> agregar(@RequestBody Producto producto) {
// Validación de datos recibidos
if (producto.getNombre() == null || producto.getNombre().isBlank()
producto.getPrecio() == null || producto.getPrecio() <= 0
producto.getExistencia() == null || producto.getExistencia
return ResponseEntity.badRequest().body("Datos de producto no
}
Producto nuevoProducto = datos.agregar(producto);
return ResponseEntity.status(HttpStatus.CREATED).body(nuevoProduct
}
}
arting Servlet engine: [Apache Tomcat/11.0.24]
2026-10-12T18:03:32.799-06:00 INFO 23472 --- [soainventario] [restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1508 ms
2026-10-12T18:03:32.971-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.b.w.a.WelcomePageHandlerMapping : Adding welcome page: class path resource [static/index.html]
2026-10-12T18:03:33.400-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-10-12T18:03:33.408-06:00 INFO 23472 --- [soainventario] [restartedMain] c.u.s.SoainventarioApplication : Started SoainventarioApplication in 2.922 seconds (process running for 3.697)
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-10-12T18:03:54.990-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
Live Share
Java: Ready
Select Postgres Server
Ln 38, Col 2
Spaces: 4
UTF-8
CRLF
Java
Prettier


File Edit Selection View Go Run
soainventario
Explorer
soainventario
.github
.mvn
.vscode
src
main
java \ com \ uvm \ soainventario
controlador
InventarioController.java
ProductoController.java
modelo
Producto.java
servicio
DatosProductos.java
SoainventarioApplication.java
resources
static
index.html
templates
application.properties
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
Outline
Timeline
Logical Structure
Java Projects
Maven
Producto.java
DatosProductos.java
ProductoController.java
src > main > java > com > uvm > soainventario > modelo > Producto.java > Producto
package com.uvm.soainventario.modelo;
public class Producto {
private Integer id;
private String nombre;
private Double precio;
private Integer existencia;
public Producto() {
}
public Producto(Integer id, String nombre, Double precio, Integer exis
this.id = id;
this.nombre = nombre;
this.precio = precio;
this.existencia = existencia;
}
public Integer getId() {
return id;
}
public void setId(Integer id) {
this.id = id;
}
public String getNombre() {
return nombre;
}
public void setNombre(String nombre) {
this.nombre = nombre;
}
public Double getPrecio() {
return precio;
}
public void setPrecio(Double precio) {
this.precio = precio;
}
public Integer getExistencia() {
return existencia;
}
public void setExistencia(Integer existencia) {
this.existencia = existencia;
}
}
arting Servlet engine: [Apache Tomcat/11.0.24]
2026-10-12T18:03:32.799-06:00 INFO 23472 --- [soainventario] [restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1508 ms
2026-10-12T18:03:32.971-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.b.w.a.WelcomePageHandlerMapping : Adding welcome page: class path resource [static/index.html]
2026-10-12T18:03:33.400-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-10-12T18:03:33.408-06:00 INFO 23472 --- [soainventario] [restartedMain] c.u.s.SoainventarioApplication : Started SoainventarioApplication in 2.922 seconds (process running for 3.697)
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-10-12T18:03:54.990-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
Live Share
Java: Ready
Select Postgres Server
Ln 50, Col 2
Spaces: 4
UTF-8
CRLF
Java
Prettier

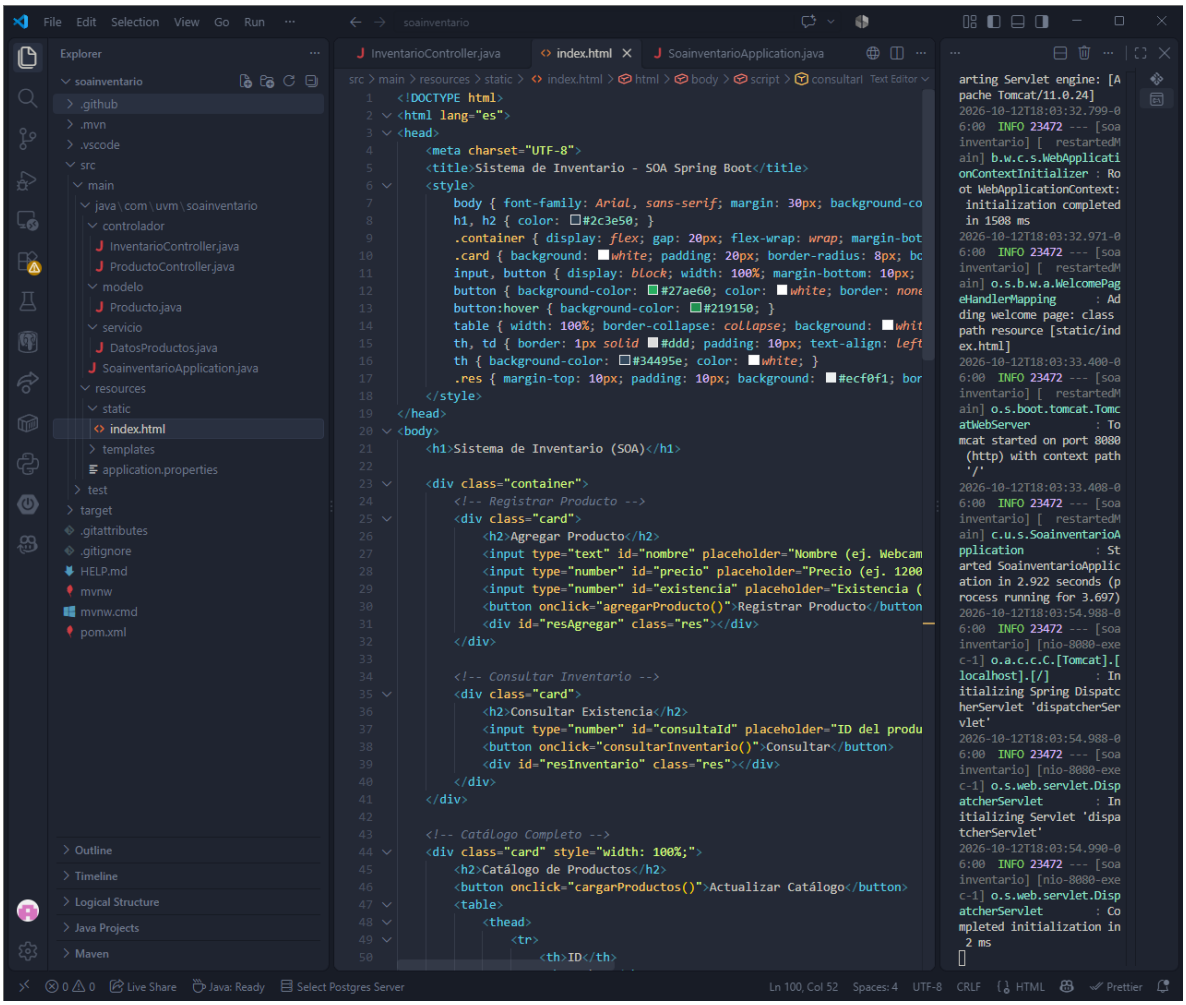

File Edit Selection View Go Run
soainventario
Explorer
soainventario
.github
.mvn
.vscode
src
main
java \ com \ uvm \ soainventario
controlador
InventarioController.java
ProductoController.java
modelo
Producto.java
servicio
DatosProductos.java
SoainventarioApplication.java
resources
static
index.html
templates
application.properties
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
Outline
Timeline
Logical Structure
Java Projects
Maven
Producto.java
DatosProductos.java
ProductoController.java
src > main > java > com > uvm > soainventario > servicio > DatosProductos.java > Language Support for
1 package com.uvm.soainventario.servicio;
2
3 import com.uvm.soainventario.modelo.Producto;
4 import org.springframework.stereotype.Service;
5
6 import java.util.ArrayList;
7 import java.util.List;
8 import java.util.Optional;
9 import java.util.concurrent.atomic.AtomicInteger;
10
11 @Service
12 public class DatosProductos {
13 private final List<Producto> productos = new ArrayList<>();
14 private final AtomicInteger idGenerator = new AtomicInteger(1);
15
16 public DatosProductos() {
17 // Registro de productos iniciales para pruebas
18 productos.add(new Producto(idGenerator.getAndIncrement(), nombre:
19 productos.add(new Producto(idGenerator.getAndIncrement(), nombre:
20 }
21
22 public List<Producto> consultar() {
23 return productos;
24 }
25
26 public Producto agregar(Producto producto) {
27 producto.setId(idGenerator.getAndIncrement());
28 productos.add(producto);
29 return producto;
30 }
31
32 public Optional<Producto> buscar(int id) {
33 return productos.stream()
34 .filter(p -> p.getId() == id)
35 .findFirst();
36 }
37 }
arting Servlet engine: [Apache Tomcat/11.0.24]
2026-10-12T18:03:32.799-06:00 INFO 23472 --- [soainventario] [restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1508 ms
2026-10-12T18:03:32.971-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.b.w.a.WelcomePageHandlerMapping : Adding welcome page: class path resource [static/index.html]
2026-10-12T18:03:33.400-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-10-12T18:03:33.408-06:00 INFO 23472 --- [soainventario] [restartedMain] c.u.s.SoainventarioApplication : Started SoainventarioApplication in 2.922 seconds (process running for 3.697)
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-10-12T18:03:54.990-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
Live Share
Java: Ready
Select Postgres Server
Ln 37, Col 2
Spaces: 4
UTF-8
CRLF
Java
Prettier


File Edit Selection View Go Run
soainventario
Explorer
soainventario
.github
.mvn
.vscode
src
main
java \ com \ uvm \ soainventario
controlador
InventarioController.java
ProductoController.java
modelo
Producto.java
servicio
DatosProductos.java
SoainventarioApplication.java
resources
static
index.html
templates
application.properties
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
Outline
Timeline
Logical Structure
Java Projects
Maven
InventarioController.java
index.html
SoainventarioApplication.java
src > main > resources > static > index.html > html > body > script > consultarI
Text Editor
<html lang="es">
<body>
<script>
async function agregarProducto() {
const res = await fetch(`${API}/productos`, {
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ nombre, precio, existencia })
});
if (res.ok) {
const data = await res.json();
document.getElementById("resAgregar").innerText = `Registr
cargarProductos();
} else {
document.getElementById("resAgregar").innerText = "Error e
}
}
async function consultarInventario() {
const id = document.getElementById("consultaId").value;
const res = await fetch(`${API}/inventario/${id}`);
const data = await res.json();
if (res.ok) {
document.getElementById("resInventario").innerText = data.
} else {
document.getElementById("resInventario").innerText = data.
}
}
cargarProductos();
</script>
</body>
</html>
arting Servlet engine: [Apache Tomcat/11.0.24]
2026-10-12T18:03:32.799-06:00 INFO 23472 --- [soainventario] [restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1508 ms
2026-10-12T18:03:32.971-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.b.w.a.WelcomePageHandlerMapping : Adding welcome page: class path resource [static/index.html]
2026-10-12T18:03:33.400-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-10-12T18:03:33.408-06:00 INFO 23472 --- [soainventario] [restartedMain] c.u.s.SoainventarioApplication : Started SoainventarioApplication in 2.922 seconds (process running for 3.697)
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-10-12T18:03:54.990-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
Live Share
Java: Ready
Select Postgres Server
Ln 100, Col 52
Spaces: 4
UTF-8
CRLF
HTML
Prettier

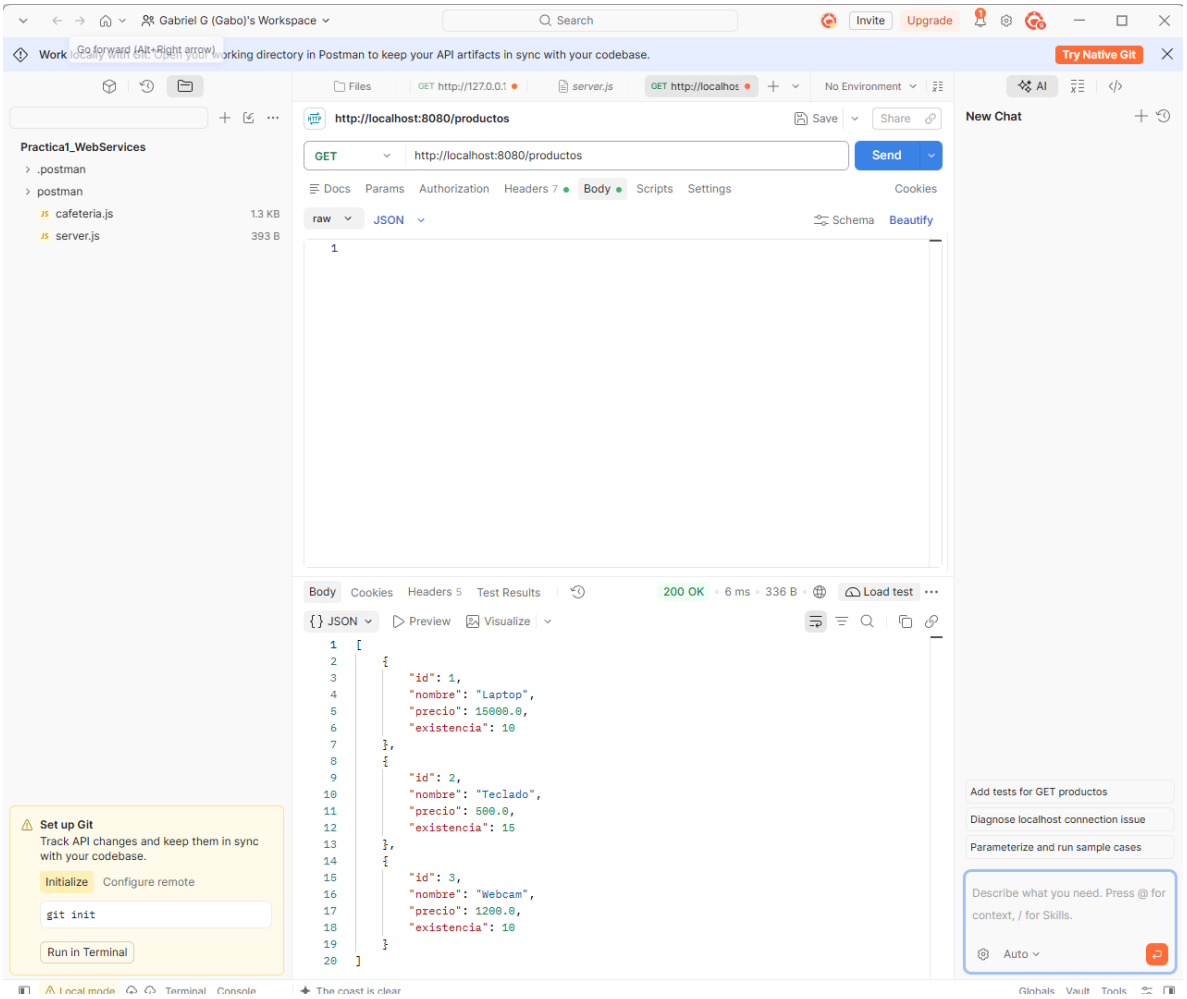

File Edit Selection View Go Run
soainventario
Explorer
soainventario
.github
.mvn
.vscode
src
main
java \ com \ uvm \ soainventario
controlador
InventarioController.java
ProductoController.java
modelo
Producto.java
servicio
DatosProductos.java
SoainventarioApplication.java
resources
static
index.html
templates
application.properties
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
Outline
Timeline
Logical Structure
Java Projects
Maven
InventarioController.java
index.html
SoainventarioApplication.java
src > main > resources > static > index.html > html > body > script > consultarI
Text Editor
<html lang="es">
<body>

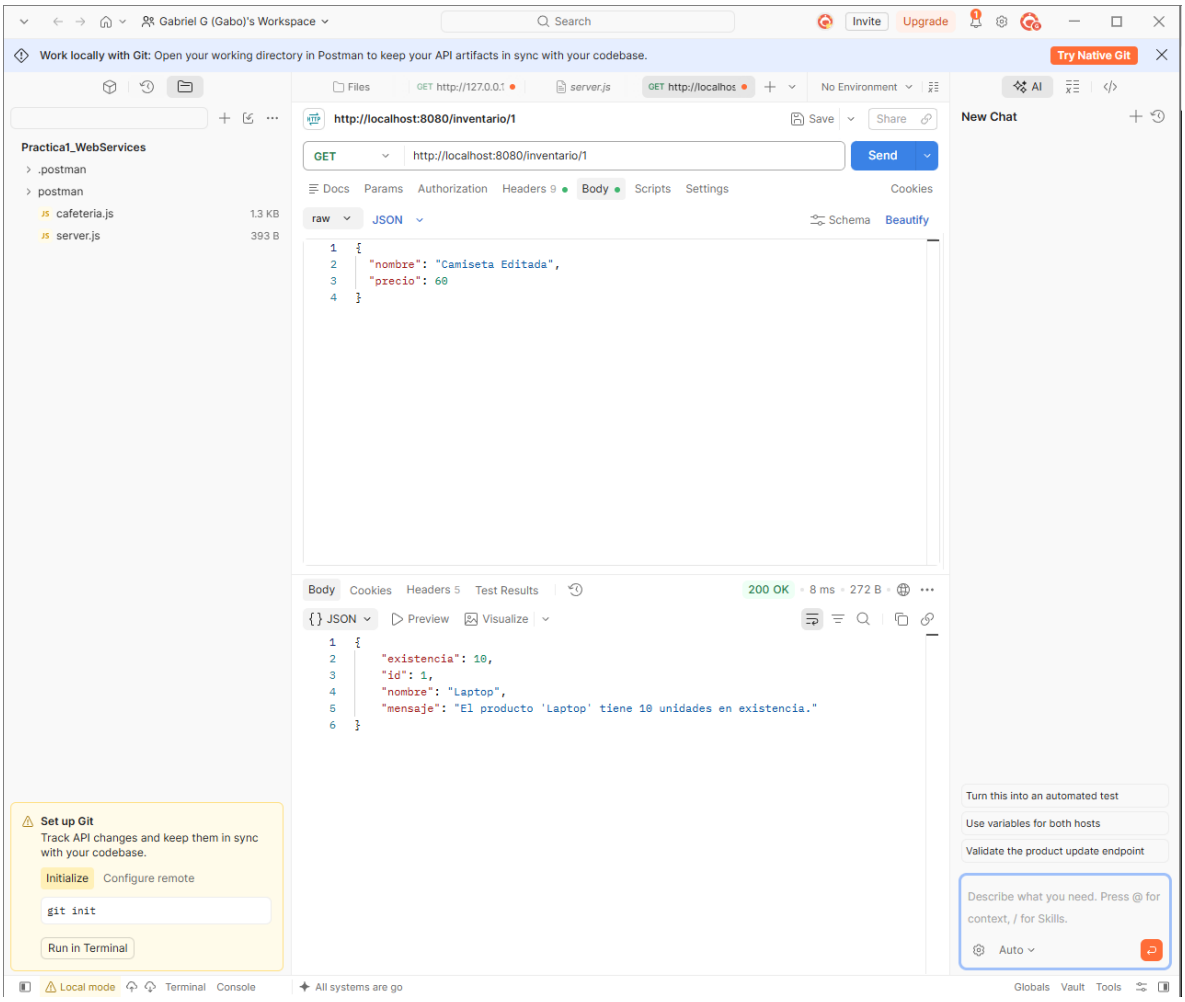
<table>
<thead>
<th>Nombre</th>
<th>Precio</th>
<th>Existencia</th>
</tr>
</thead>
<tbody id="tablaProductos"></tbody>
</table>

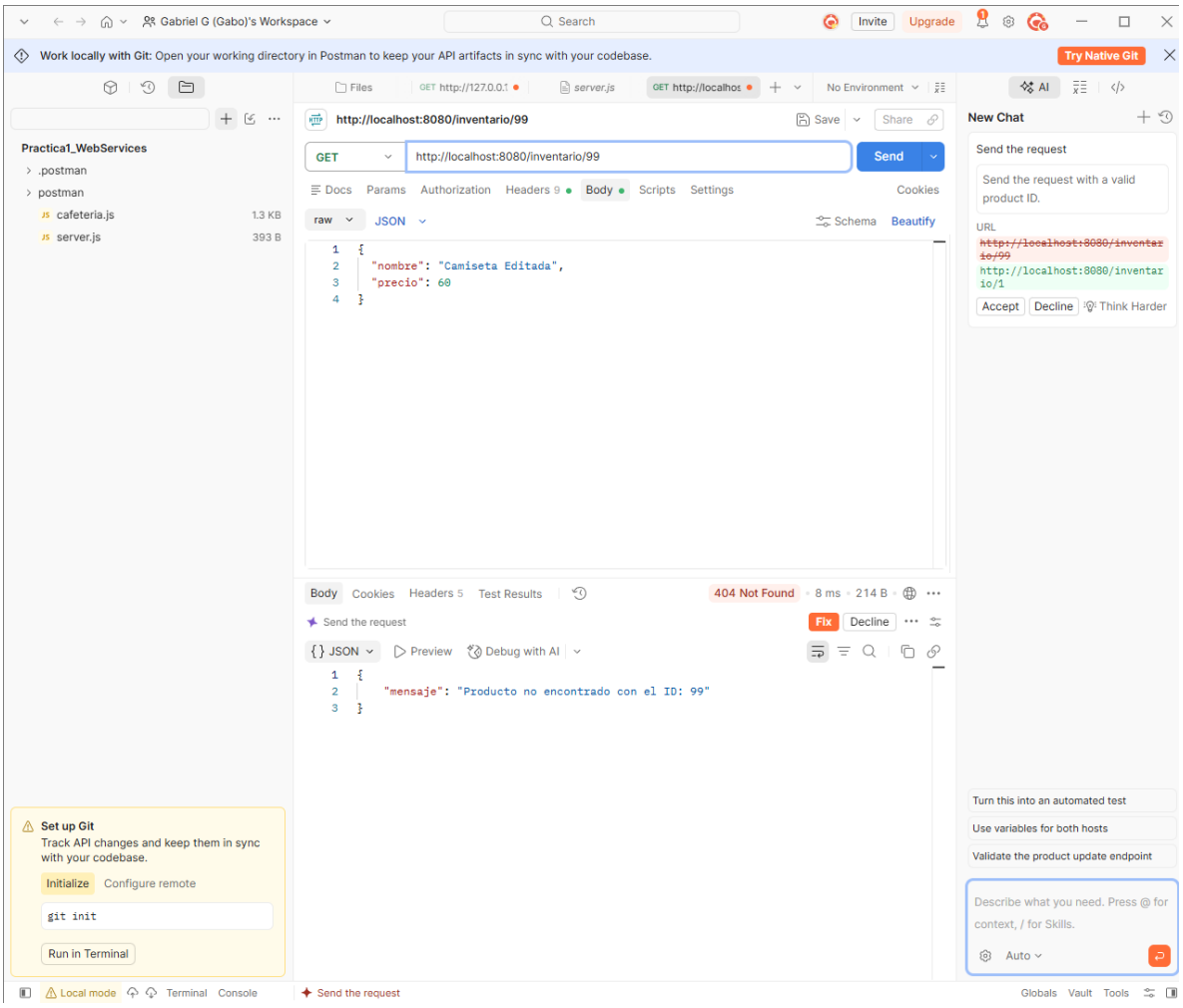
<script>
const API = "http://localhost:8080";
async function cargarProductos() {
const res = await fetch(`${API}/productos`);
const productos = await res.json();
const tbody = document.getElementById("tablaProductos");
tbody.innerHTML = "";
productos.forEach(p => {
tbody.innerHTML += `<tr>
<td>${p.id}</td>
<td>${p.nombre}</td>
<td>$${p.precio}</td>
<td>${p.existencia}</td>
</tr>`;
});
}
async function agregarProducto() {
const nombre = document.getElementById("nombre").value;
const precio = parseFloat(document.getElementById("precio").va
const existencia = parseInt(document.getElementById("existenci
const res = await fetch(`${API}/productos`, {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ nombre, precio, existencia })
});
if (res.ok) {
const data = await res.json();
document.getElementById("resAgregar").innerText = `Registr
cargarProductos();
} else {
document.getElementById("resAgregar").innerText = "Error e
}
arting Servlet engine: [Apache Tomcat/11.0.24]
2026-10-12T18:03:32.799-06:00 INFO 23472 --- [soainventario] [restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1508 ms
2026-10-12T18:03:32.971-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.b.w.a.WelcomePageHandlerMapping : Adding welcome page: class path resource [static/index.html]
2026-10-12T18:03:33.400-06:00 INFO 23472 --- [soainventario] [restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-10-12T18:03:33.408-06:00 INFO 23472 --- [soainventario] [restartedMain] c.u.s.SoainventarioApplication : Started SoainventarioApplication in 2.922 seconds (process running for 3.697)
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-12T18:03:54.988-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-10-12T18:03:54.990-06:00 INFO 23472 --- [soainventario] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
Live Share
Java: Ready
Select Postgres Server
Ln 100, Col 52
Spaces: 4
UTF-8
CRLF
HTML
Prettier

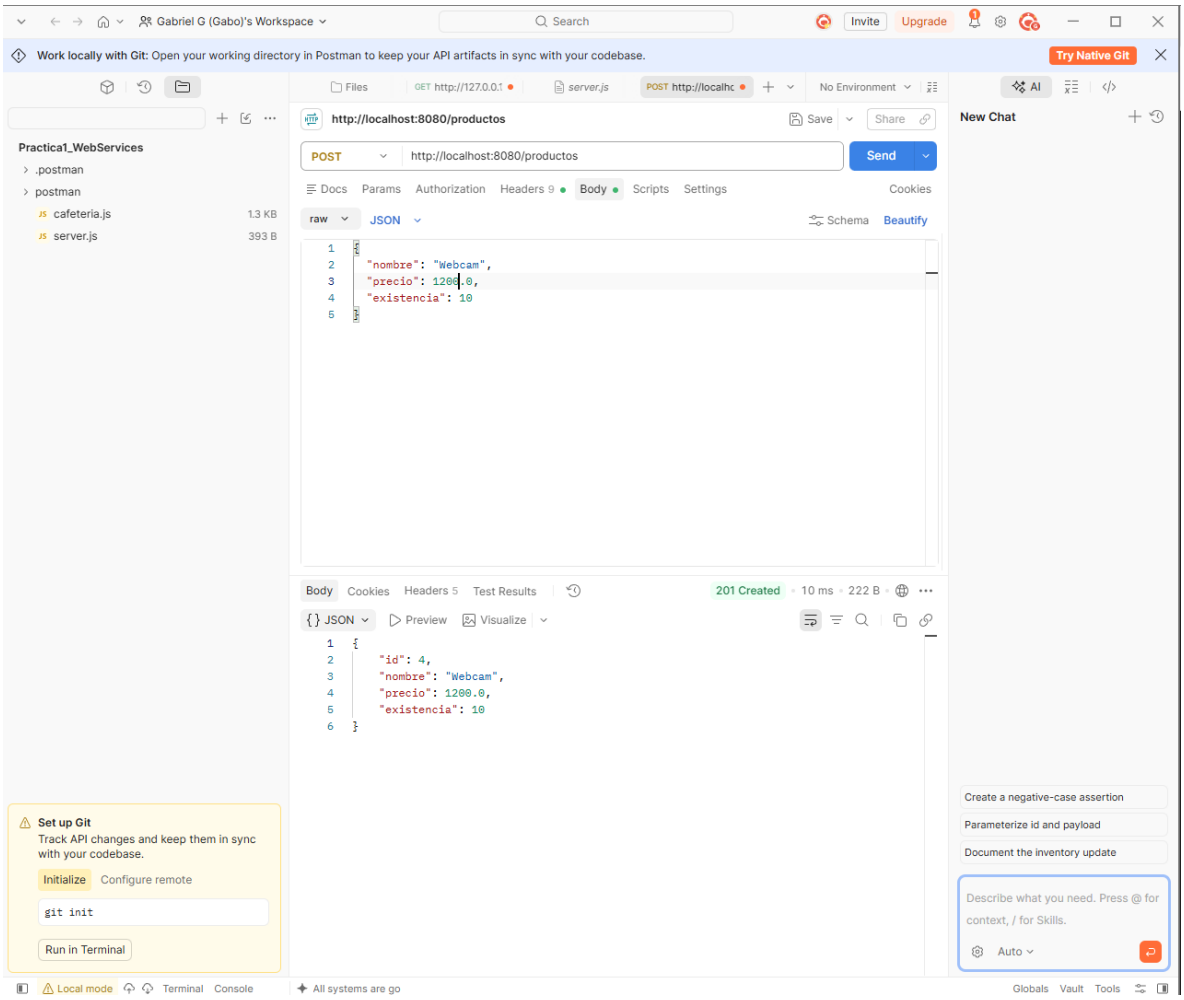


Capturas Postman


Gabriel G (Gabo)'s Workspace
Search
Invite
Upgrade
Try Native Git
Practica1_WebServices
.postman
postman
cafeteria.js 1.3 KB
server.js 393 B
http://localhost:8080/productos
Save
Share
GET
http://localhost:8080/productos
Send
Docs Params Authorization Headers 7 Body Scripts Settings Cookies
raw JSON
Schema Beautify
Body Cookies Headers 5 Test Results
200 OK 6 ms 336 B
Load test
JSON Preview Visualize
[
{
"id": 1,
"nombre": "Laptop",
"precio": 15000.0,
"existencia": 10
},
{
"id": 2,
"nombre": "Teclado",
"precio": 500.0,
"existencia": 15
},
{
"id": 3,
"nombre": "Webcam",
"precio": 1200.0,
"existencia": 10
}
]
Set up Git
Track API changes and keep them in sync with your codebase.
Initialize
Configure remote
git init
Run in Terminal
New Chat
Add tests for GET productos
Diagnose localhost connection issue
Parameterize and run sample cases
Describe what you need. Press @ for context, / for Skills.
Auto


Gabriel G (Gabo)'s Workspace
Search
Invite
Upgrade
Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.
Try Native Git
Practica1_WebServices
.postman
postman
cafeteria.js 1.3 KB
server.js 393 B
http://localhost:8080/inventario/1
Save
Share
GET
http://localhost:8080/inventario/1
Send
Docs Params Authorization Headers 9 Body Scripts Settings Cookies
raw JSON
Schema Beautify
{
"nombre": "Camiseta Editada",
"precio": 60
}
Body Cookies Headers 5 Test Results
200 OK 8 ms 272 B
JSON Preview Visualize
{
"existencia": 10,
"id": 1,
"nombre": "Laptop",
"mensaje": "El producto 'Laptop' tiene 10 unidades en existencia."
}
New Chat
Turn this into an automated test
Use variables for both hosts
Validate the product update endpoint
Describe what you need. Press @ for context, / for Skills.
Auto
Set up Git
Track API changes and keep them in sync with your codebase.
Initialize
Configure remote
git init
Run in Terminal
Local mode
Terminal
Console
All systems are go
Globals
Vault
Tools


Gabriel G (Gabo)'s Workspace
Search
Invite
Upgrade
Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.
Try Native Git
Practica1_WebServices
.postman
postman
cafeteria.js 1.3 KB
server.js 393 B
http://localhost:8080/inventario/99
Save
Share
GET
http://localhost:8080/inventario/99
Send
Docs Params Authorization Headers 9 Body Scripts Settings Cookies
raw JSON
Schema Beautify
{
"nombre": "Camiseta Editada",
"precio": 60
}
Body Cookies Headers 5 Test Results
404 Not Found 8 ms 214 B
Send the request
Fix Decline
JSON Preview Debug with AI
{
"mensaje": "Producto no encontrado con el ID: 99"
}
New Chat
Send the request
Send the request with a valid product ID.
URL
http://localhost:8080/inventario/99
http://localhost:8080/inventario/1
Accept Decline Think Harder
Turn this into an automated test
Use variables for both hosts
Validate the product update endpoint
Describe what you need. Press @ for context, / for Skills.
Auto
Set up Git
Track API changes and keep them in sync with your codebase.
Initialize Configure remote
git init
Run in Terminal
Local mode Terminal Console
Send the request
Globals Vault Tools



PREGUNTAS

1. ¿Qué diferencia existe entre el servicio de productos y el servicio de inventario?

El servicio de productos administra la información de los artículos, permitiendo consultarlos y registrarlos. El servicio de inventario se encarga de verificar la cantidad disponible de un producto específico mediante su identificador. Cada uno cumple una función distinta dentro del sistema.

2. ¿Qué función cumple @RestController?

@RestController indica que una clase actuará como controlador de servicios web en Spring Boot. Permite recibir solicitudes HTTP y devolver respuestas directamente en formato JSON u otro formato compatible con una API REST.

3. ¿Por qué los dos controladores pueden utilizar los mismos datos?

Los dos controladores utilizan la misma clase de datos compartida, por lo que acceden a la misma información almacenada. Esto evita la duplicación de registros y mantiene la consistencia de los datos en todo el sistema.

4. ¿Qué ventajas tiene separar las responsabilidades en servicios?

Cada servicio se enfoca en una tarea específica, lo que facilita el mantenimiento del código, la detección de errores y la realización de cambios. También permite reutilizar funcionalidades y mejorar la organización general de la aplicación.

5. ¿Cuál es la diferencia entre SOA y REST?

SOA es una arquitectura basada en servicios independientes que colaboran entre sí para cumplir procesos de negocio. REST es un estilo arquitectónico utilizado para crear servicios web mediante HTTP. SOA define la estructura general de los servicios y REST proporciona una forma de implementarlos y consumirlos.

6. ¿Qué semejanzas encontraron entre Express.js y Spring Boot?

Ambos frameworks permiten desarrollar APIs y servicios web. Los dos utilizan rutas para atender peticiones HTTP, trabajan fácilmente con datos JSON y facilitan la organización del proyecto mediante componentes encargados de manejar las solicitudes y respuestas. Esto simplifica el desarrollo de aplicaciones backend.